

VisionLab

Image Processing and Computer Vision Analysis Tool

Project Report

Course: Computer Vision (VITyarthi Flipped Course Evaluation)

Submitted by: Dev

B.Tech CSE

2. Introduction

VisionLab is an interactive Streamlit application built to demonstrate the core Computer Vision concepts covered across the course syllabus. Rather than isolated notebook snippets, VisionLab brings image enhancement, feature detection, and segmentation/clustering algorithms together into a single, cohesive tool that lets a user upload an image and immediately see the effect of each classical CV algorithm, with adjustable parameters and side-by-side before/after comparisons.

The project deliberately restricts itself to classical, well-understood computer vision techniques implemented directly with OpenCV and NumPy (with scikit-learn used only for K-Means clustering). No deep learning, database, authentication, or external API is used, keeping the implementation transparent, inspectable, and appropriate in scope for a course project.

3. Problem Statement

Students learning Computer Vision typically encounter core algorithms in isolated lecture demos or notebook snippets, with no single tool that lets them apply these algorithms interactively to their own images and build intuition for how parameters affect results. VisionLab addresses this by providing a self-contained, interactive application covering the syllabus's classical CV techniques, usable without writing any code.

4. Functional Requirements

VisionLab implements three major functional modules, each with a clear input/output structure and a logical, linear user workflow (upload/select image → choose module → choose operation → adjust parameters → view result → download).

4.1 Image Enhancement

- Flip (horizontal / vertical / both)
- Contrast adjustment
- Noise removal (Gaussian, median, bilateral)
- Linear transformation
- Log transformation
- Power-law (gamma) transformation
- Histogram equalization (grayscale and color-preserving)
- Basic morphology: erosion, dilation, opening, closing

4.2 Feature Detection

- Canny edge detection

- Corner detection (Shi-Tomasi and Harris)
- Hough line detection (probabilistic Hough transform)

4.3 Segmentation & Clustering

- Watershed segmentation (marker-based, distance-transform driven)
- K-Means clustering (color-based image segmentation)

5. Non-Functional Requirements

Requirement	How VisionLab addresses it
Performance	Uploaded images are automatically downscaled beyond a safe dimension so every operation stays
Reliability	All algorithms are deterministic, well-established OpenCV/NumPy/scikit-learn implementations.
Usability	Simple sidebar + tabbed navigation, sensible parameter defaults, immediate visual feedback.
Maintainability	Clean modular package structure — one responsibility per file, docstrings throughout.
Error handling / Validation	A dedicated validation layer checks every file and parameter before processing; invalid input produ
Resource efficiency	Images are processed as in-memory NumPy arrays with defensive copies only where needed, and

6. System Architecture

VisionLab follows a simple three-layer architecture: a Streamlit presentation layer (app.py), a set of CV processing modules (one file per functional area), and a shared utilities layer (validation, image I/O, and visualization) used by both the UI and the processing modules.

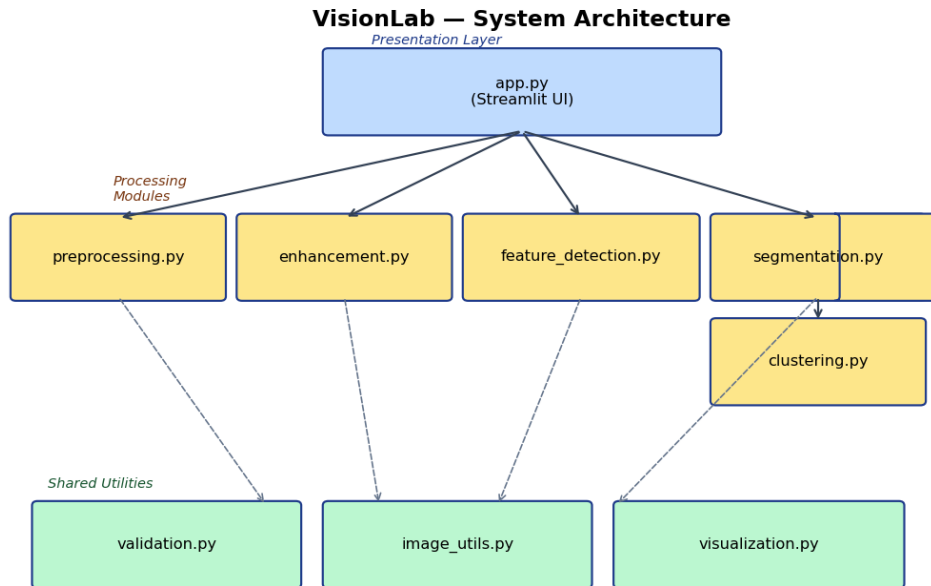


Figure 1: VisionLab system architecture

7. Design Diagrams

Full Use Case, Class/Component, and Sequence diagrams (in Mermaid format, rendering natively on GitHub) are provided in **docs/diagrams.md** in the project repository, alongside the Process Flow / Workflow diagram. A summary of each is given below.

7.1 Use Case Diagram (summary)

The single actor, *User*, can: upload an image, select a sample image, apply an enhancement operation, apply feature detection, apply segmentation/clustering, view the before/after comparison, view a histogram comparison, and download the processed result. Every processing use case includes the "view before/after comparison" use case.

7.2 Class / Component Diagram (summary)

VisionLab is implemented as a functional module library rather than a class hierarchy. The key components are: **app** (UI orchestration), **preprocessing** (image loading), **enhancement**, **feature_detection**, **segmentation**, and **clustering** (the CV modules), and the shared **validation**, **image_utils**, and **visualization** utilities that every module depends on.

7.3 Sequence Diagram (summary)

A typical interaction — e.g. applying gamma transformation — flows as: the UI passes the uploaded file to *preprocessing*, which validates and decodes it; the UI then passes the image and the user's gamma value to *enhancement.gamma_transform*, which validates the parameter, applies a LUT-based gamma

correction, and returns the result; the UI renders a side-by-side comparison via *visualization* and offers a download.

7.4 Process Flow / Workflow Diagram (summary)

The end-to-end workflow is: choose an image source (upload or sample) → validate/load → choose a module and operation → adjust parameters → process → display comparison (and histogram, where relevant) → download. Invalid input or out-of-range parameters loop back to the parameter-adjustment step with a clear error message rather than crashing the app.

8. Design Decisions & Rationale

- **Functional modules over classes:** Each CV operation is a standalone, pure function (input image + parameters → output image). This keeps every operation independently testable and easy to reason about, and matches how the syllabus presents each algorithm as a discrete technique.
- **Centralized validation:** A single validation.py module owns all parameter-range and file-validation logic, raising one custom VisionLabError type. This means every module and the UI can handle errors uniformly, and validation rules are defined in exactly one place.
- **Defensive copies over mutation:** Every processing function returns a new array rather than mutating its input in place. This matters specifically because Streamlit reruns the entire script on every widget interaction, so cached/session images must never be silently altered.
- **YCrCb histogram equalization for color images:** Rather than equalizing each BGR channel independently (which distorts color balance), the luminance channel alone is equalized in YCrCb space, preserving the image's original colors while still improving contrast.
- **scikit-learn confined to clustering.py:** Per the project's tech-stack constraints, scikit-learn is used only for K-Means; all other algorithms use OpenCV/NumPy directly, keeping the dependency footprint minimal and each module's purpose unambiguous.
- **Automatic image downscaling:** Images above a safe maximum dimension are downscaled before processing (preserving aspect ratio) to keep the app responsive on large uploads without requiring the user to resize images themselves.

9. Implementation Details

The codebase is organized into three layers, described below.

9.1 Utilities (utils/)

- **validation.py** — VisionLabError exception plus validators for file extension, file size, decoded image arrays, numeric parameters, odd kernel sizes, string choices, and K-Means cluster count.
- **image_utils.py** — decoding uploaded bytes to a BGR NumPy array, safe downscaling, RGB/gray/BGR color-space conversions, PNG/JPG encoding for download, and image dimension lookup.
- **visualization.py** — Matplotlib figure builders: side-by-side comparison, single and comparative histograms, single-image display, and explicit figure cleanup to avoid memory buildup across Streamlit reruns.

9.2 Processing Modules (modules/)

- **preprocessing.py** — ties validation and image_utils together into the load → validate → resize pipeline used by both file uploads and bundled sample images.

- **enhancement.py** — flip, contrast, noise removal (three filter types), linear/log/gamma point transforms, histogram equalization, and morphology (four operations), each validating its own parameters.
- **feature_detection.py** — Canny edge detection returns a binary edge map directly; corner detection (Shi-Tomasi via `goodFeaturesToTrack`, Harris via `cornerHarris`) and Hough line detection draw their results onto a copy of the original image so results are immediately viewable.
- **segmentation.py** — implements the standard OpenCV marker-based watershed recipe: Otsu threshold, morphological opening to remove noise, distance transform to find sure foreground, connected components for initial markers, and `cv2.watershed` to grow boundaries.
- **clustering.py** — K-Means over the image's (B, G, R) pixel values via `scikit-learn`, offered both as a recolored segmentation (`kmeans_segment`) and a raw label map (`kmeans_cluster_map`).

9.3 Application (app.py)

`app.py` is intentionally thin: it wires Streamlit widgets to the module functions above and renders results via the visualization helpers. It contains no image-processing logic of its own, keeping the UI layer and the algorithmic layer cleanly separated.

10. Screenshots / Results

The figures below were generated directly from VisionLab's own modules (not mocked), run against a synthetic test image containing shapes, gradients, and injected noise, to demonstrate each operation's real output.

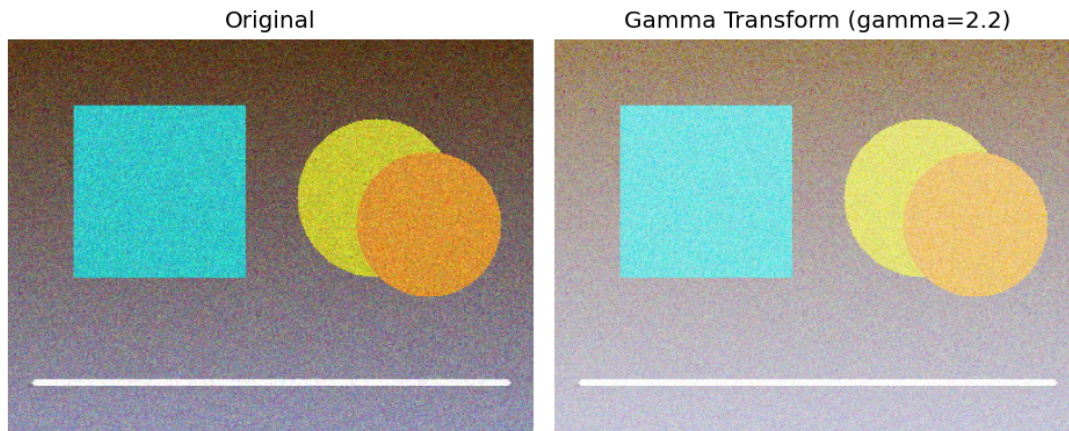


Figure 2: Gamma transformation (Image Enhancement)

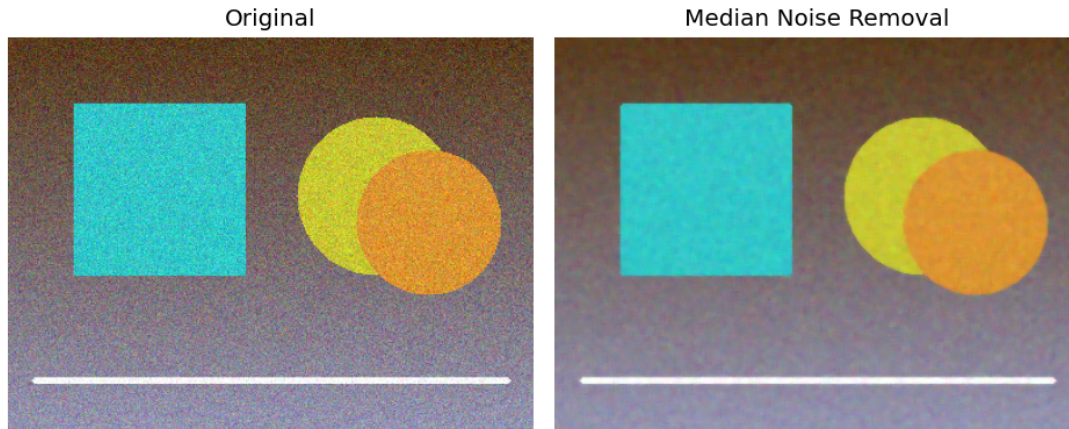


Figure 3: Median noise removal (Image Enhancement)

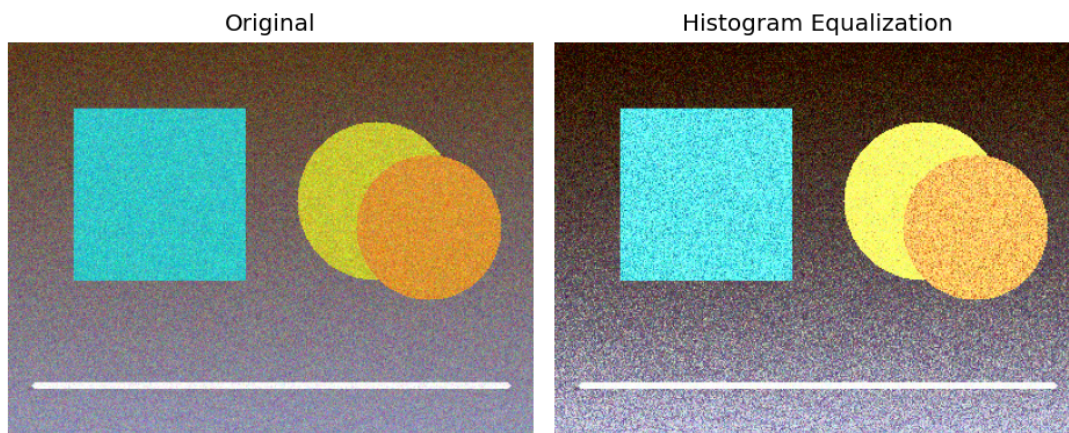


Figure 4: Histogram equalization (Image Enhancement)

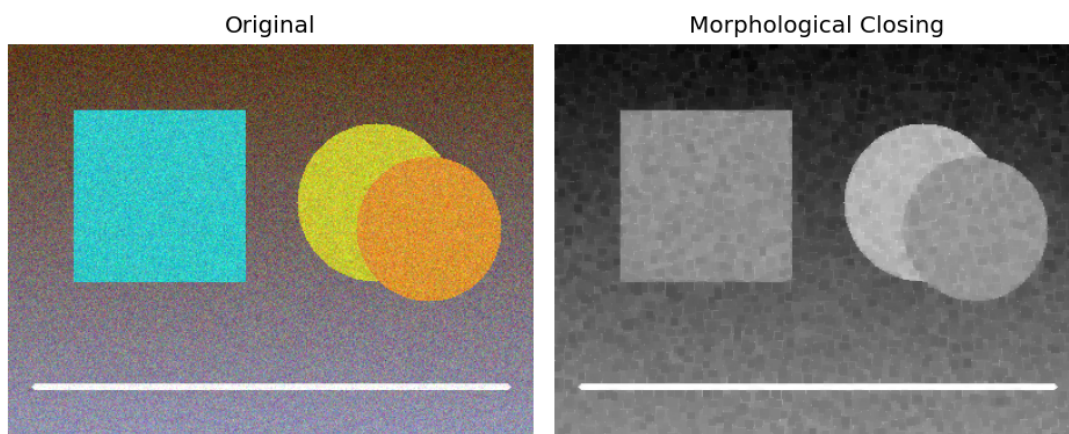


Figure 5: Morphological closing (Image Enhancement)

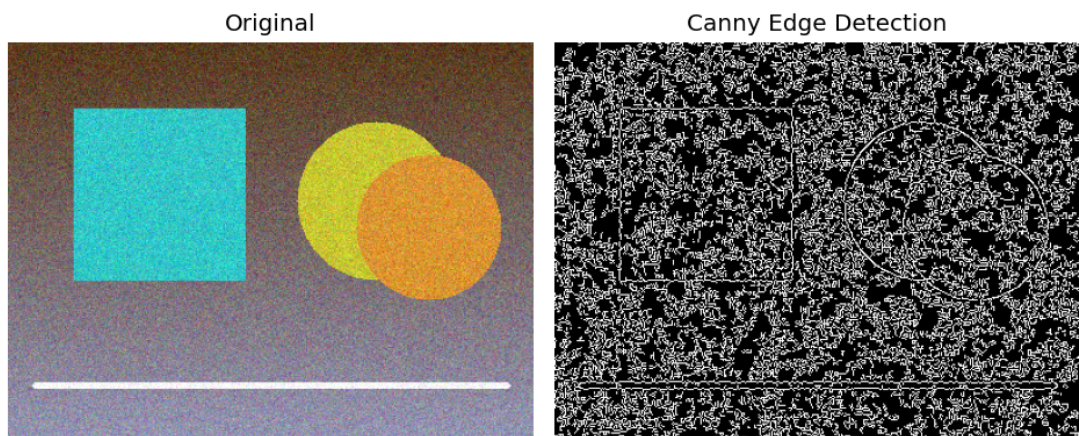


Figure 6: Canny edge detection (Feature Detection)

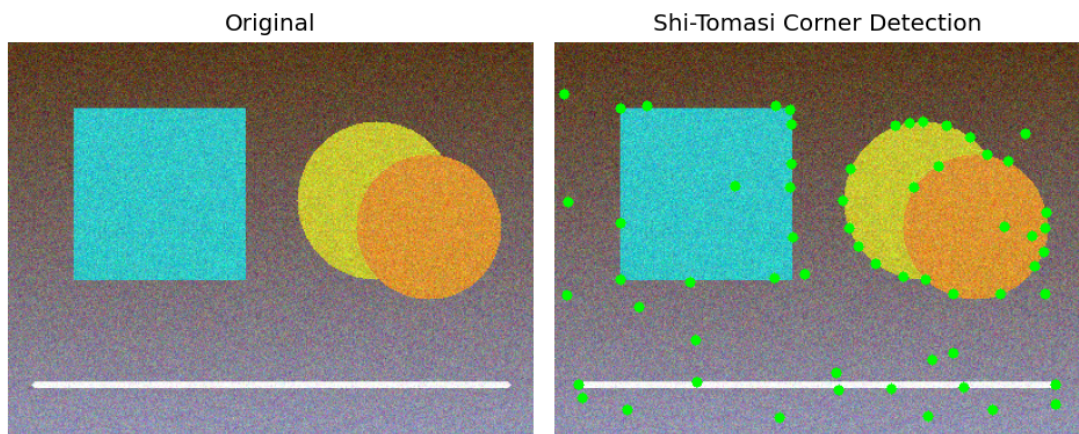


Figure 7: Shi-Tomasi corner detection (Feature Detection)

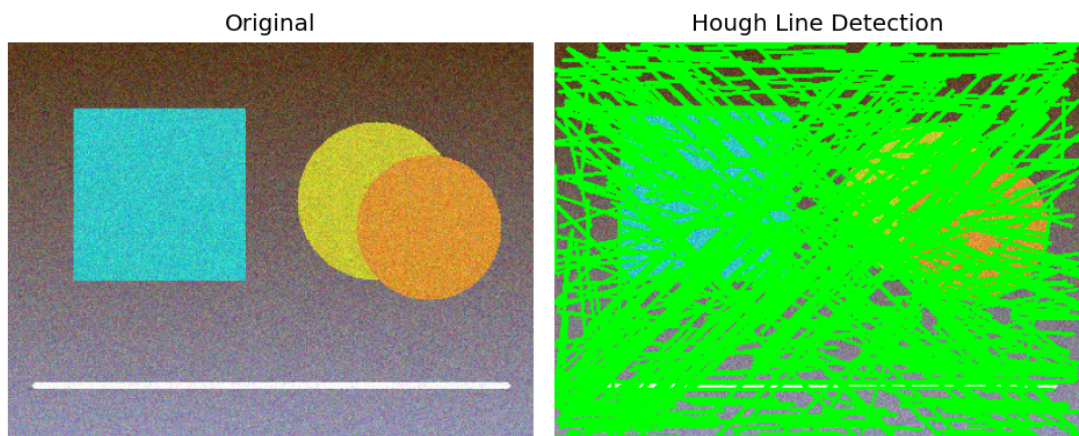


Figure 8: Hough line detection (Feature Detection)

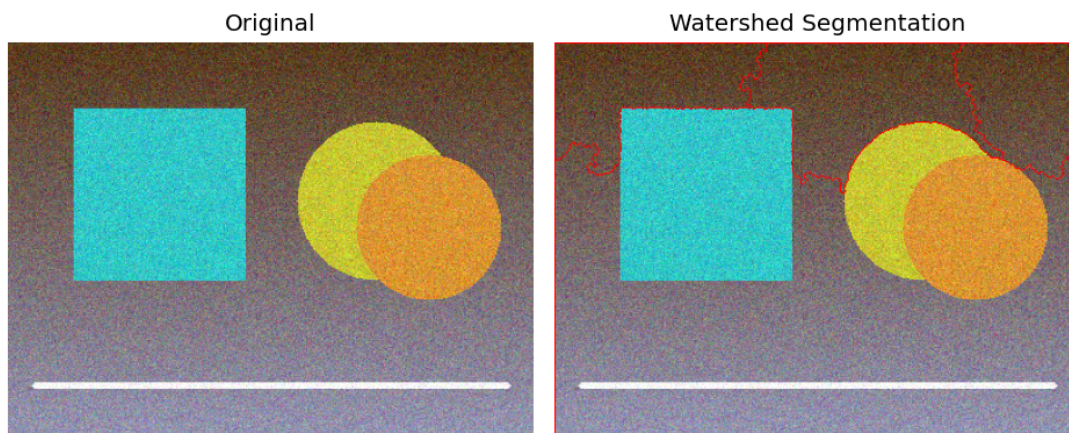


Figure 9: Watershed segmentation (Segmentation & Clustering)

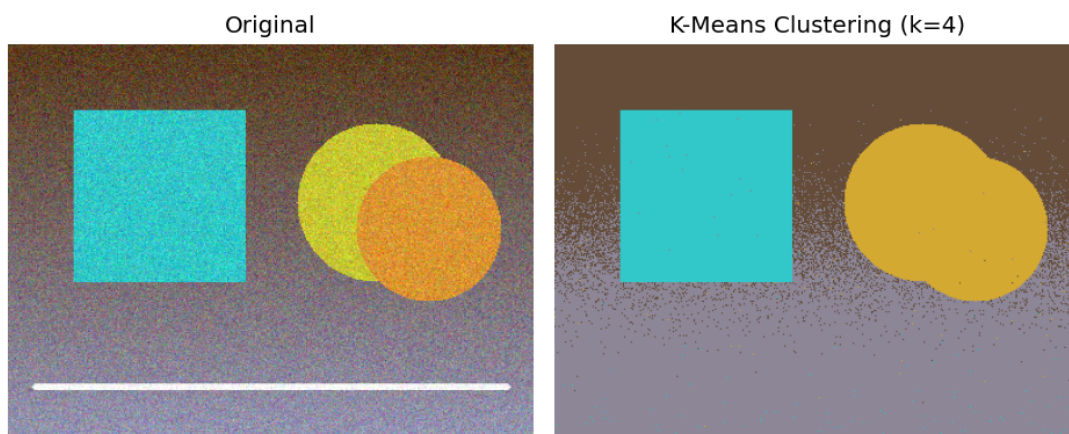


Figure 10: K-Means clustering, k=4 (Segmentation & Clustering)

11. Testing Approach

VisionLab uses pytest for unit testing, with a dedicated test file per module. Tests use synthetically generated images (solid shapes, color blocks, blank/noise images) rather than external files, so the suite runs quickly and deterministically in any environment.

Test file	Tests	Focus
test_validation.py	27	File/parameter validation, all error paths
test_enhancement.py	36	All 8 enhancement operations: shape/dtype checks, identity cases, edge cases
test_feature_detection.py	17	Canny, both corner-detection methods, Hough lines, on shapes vs. blank images
test_segmentation.py	7	Watershed on touching shapes, blank-image edge case, no-mutation check
test_clustering.py	9	K-Means cluster-count recovery, reproducibility, grayscale input
Total	96	All passing

Each test module covers: correct output shape/dtype for valid input, at least one behaviorally meaningful assertion (e.g. erosion shrinks a white region, K-Means on a 3-color image recovers at most 3 unique colors, Hough lines are found on a line image but not a blank one), grayscale-input handling, and rejection of invalid parameters via VisionLabError.

12. Challenges Faced

- **Watershed on a blank image:** initial assumption was that a blank/uniform image would produce zero boundary pixels. In practice, cv2.watershed always marks the image border as boundary (-1) by convention, even with no real foreground. The corresponding test was corrected to check for a graceful, crash-free result and correct output shape rather than an exact zero-boundary count — a good reminder to verify assumptions against the actual library behavior rather than intuition.
- **Keeping color information through histogram equalization:** naively equalizing each BGR channel independently shifts the color balance of the image. Equalizing only the luminance (Y) channel in YCrCb space solves this while still achieving the contrast improvement.
- **Balancing flexibility with input safety:** every user-adjustable parameter (kernel sizes, thresholds, cluster counts) needed a sensible allowed range that is wide enough to be useful but narrow enough to avoid pathological runtimes (e.g. capping K-Means cluster count and max Hough vote threshold).

13. Learnings & Key Takeaways

- A small, centralized validation layer dramatically simplifies error handling across an entire application — every module gets consistent, user-friendly error messages for free.
- Classical computer vision techniques (thresholding, distance transforms, connected components) remain powerful and interpretable building blocks, and watershed segmentation in particular showed how several simple steps compose into a much more capable algorithm.

- Writing tests against synthetic, purpose-built images (rather than trying to test against arbitrary real photos) makes it possible to assert specific, meaningful outcomes (e.g. "exactly 3 colors recovered") instead of only checking that code runs without crashing.
- Streamlit's rerun-on-every-interaction model has real implications for state management — processing functions must never mutate their input in place, or a later interaction can silently corrupt an earlier result.

14. Future Enhancements

- Add a batch-processing mode to apply the same operation across multiple uploaded images at once.
- Allow chaining multiple operations together (e.g. denoise → then Canny) within a single pipeline, with the ability to save and reuse a pipeline configuration.
- Add additional segmentation techniques (e.g. GrabCut, mean-shift) to broaden the Segmentation & Clustering module.
- Add a side-by-side parameter-sweep view (e.g. show gamma at several values at once) for faster visual comparison.

15. References

- OpenCV Documentation — <https://docs.opencv.org/>
- NumPy Documentation — <https://numpy.org/doc/>
- Streamlit Documentation — <https://docs.streamlit.io/>
- scikit-learn KMeans Documentation — <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>
- Matplotlib Documentation — <https://matplotlib.org/stable/>
- OpenCV Watershed Tutorial (Image Segmentation) — https://docs.opencv.org/4.x/d3/db4/tutorial_py_watershed.html
- VITyarthi Computer Vision course materials (syllabus reference for module mapping).